

Computer Science. Internal Assessment

Car Management System

Word Count: 1720

Criterion A - Problem Specification

Problem Scenario

My uncle operates a second-hand car dealership in Pakistan. Having recently started this business, he currently manages a modest inventory of 30 vehicles. Customers either visit the dealership in person or call to inquire about available cars. To track these vehicles, he relies on an Excel spreadsheet, which proves to be both time-consuming and inefficient, particularly when searching for cars based on specific customer requirements. As his inventory continues to grow, this manual system has become increasingly inefficient. To improve the management of his stock, I am planning to develop an application designed to streamline inventory management. This solution will facilitate more efficient searching and tracking of cars. Additionally, I am planning to integrate a voice recognition system into the application. This feature will allow for faster and more convenient record entry, minimizing the need for manual typing and further enhancing operational efficiency.

Computational Context

I am considering developing a standalone solution, as it will be used exclusively by my uncle and therefore does not require internet connectivity. This standalone approach is also optimal due to its adherence to the Model-View-Controller (MVC) pattern, which ensures a clear separation of data handling, user interface, and application logic. I plan to use Python for this project due to its simplicity, readability, and concise syntax. Additionally, Python offers a rich collection of libraries that will be instrumental in development. Specifically, I will utilize Tkinter for creating the graphical user interface, SQLite for database management, the Bokeh library for data visualization, and the speech_recognition and sounddevice libraries for voice recognition and processing. These tools will facilitate rapid development and robust functionality. Furthermore, Python's cross-platform compatibility will ensure the application operates smoothly across various operating systems.

Words: 283

Success Criteria

My application will:

1. Validate data
2. Add a car record
3. Edit existing car record
4. Delete a car record
5. Search for a particular car
6. Sort cars by brand name, price and year
7. Display statistical data using a graph
8. Have a voice recognition capability

Criterion B - Planning

Decomposing Problems

1. Database Management

- **Schema Definition:**
 - Define the database schema for the cars table.
 - Fields: id (auto-generated), brand, model, color, year, price, condition.
- **CRUD Operations:**
 - **Create:** Add new records to the database.
 - **Read:** Retrieve and display records.
 - **Update:** Modify existing records.
 - **Delete:** Remove records from the database.
- **Search and Sorting:**
 - Implement search functionality for finding records based on criteria.
 - Implement sorting functionality by brand, price, and year.

2. User Interface (UI)

- **Form Design:**
 - **Input Fields:**
 - **Combo Boxes:** For Brand and Color, allowing users to select from predefined lists.
 - **Text Fields:** For Model, Year, and Price, where users can enter data manually.
 - **Condition:** Text field to display the recognized condition from voice input.
 - **Buttons:**
 - **Add:** To submit new records.
 - **Edit:** To modify existing records.
 - **Delete:** To remove selected records.
 - **Search:** To find records based on user input.
 - **Statistics:** To display data visualizations.

- **Speak Condition:** To activate voice recognition for the Condition field.

- **Display:**
 - **Record Table:** To view records from the database.
 - **Data Visualization:** To show statistics and graphical representations using Bokeh.

3. Voice Recognition

- **Voice Recording:**
 - Use sounddevice to record audio input.
- **Speech Recognition:**
 - Process recorded audio using speech_recognition to convert it to text.
- **Field Population:**
 - Automatically populate the Condition field with recognized text from voice input.

4. Data Visualization

- **Bokeh Integration:**
 - **Visualization Setup:**
 - Configure Bokeh to generate visualizations based on available data.
 - **Graph Types:** Create bar charts or other visual representations of car counts by brand, price, etc.
 - **Display:**
 - Render visualizations in a new window or integrated within the Tkinter application.

5. Error Handling and Validation

- **Input Validation:**
 - Ensure that all data entered by the user is valid and meets the required format.

- **Error Handling:**

- Provide user feedback on errors or issues during operations (e.g., invalid input, database errors).

6. Controller Logic

- **Event Handling:**

- Connect UI actions (button clicks, input changes) to the corresponding functionality in the model.

- **Data Synchronization:**

- Ensure that updates in the UI are reflected in the database and vice versa.

- **Voice Recognition Integration:**

- Manage interactions between the voice recognition component and the UI for filling in the Condition field.

Planning the computational thinking process

Week 1-2: Preparation and Initial Setup.

Task 1: Set Up Development Environment

Dependencies: None

Subtasks:

1. Planning:
 - Identify required tools and libraries.
 - Create a setup checklist.
2. Designing:
 - Plan the development environment layout.
3. Developing:
 - Install Python, Tkinter, sqlite3, Bokeh, sounddevice, and speech_recognition.
 - Set up version control (Git).
4. Testing:
 - Verify installations and configuration.
5. Evaluating:
 - Confirm readiness for development.

Week 3-4: Database Management

Task 2: Database Schema Definition

Dependencies: Completion of Development Environment setup.

Subtasks:

1. Planning:
 - Define database schema for the cars table with required fields.
2. Designing:
 - Create schema setup scripts.
3. Developing:
 - Implement schema setup scripts.
4. Testing:
 - Verify schema creation.
5. Evaluating:
 - Ensure schema meets requirements.

Resources: Database design tools, SQLite documentation.

Task 3: CRUD Operations Implementation

Dependencies: Completion of Database Schema Definition.

Subtasks:

1. Planning:
 - Define operations for Create, Read, Update, Delete.
2. Designing:
 - Design methods for each operation.
3. Developing:
 - Write methods for CRUD operations and sorting.
4. Testing:
 - Verify CRUD and sorting functions.
5. Evaluating:
 - Ensure operations meet requirements.

Resources: Python, SQLite documentation.

Week 5-6: User Interface (UI) Design and Implementation

Task 4: Form and Button Design

Dependencies: Completion of CRUD Operations Implementation.

Subtasks:

1. Planning:
 - Design layout for forms and buttons.
2. Designing:
 - Create UI mockups.
3. Developing:
 - Implement forms and buttons.
 - Add combo boxes for Brand and Color.
4. Testing:
 - Verify functionality of forms, buttons, and combo boxes.
5. Evaluating:
 - Ensure UI components meet design requirements.

Resources: Tkinter documentation, UI design tools.

Task 5: Display and Interaction

Dependencies: Completion of Form and Button Design.

Subtasks:

1. Planning:
 - Design layout for record table and data visualization.
2. Designing:
 - Create UI mockups.
3. Developing:
 - Implement record table and data visualization using Bokeh.
4. Testing:

- Verify functionality of display components.
- 5. Evaluating:
 - Ensure display components meet design requirements.

Resources: Tkinter documentation, Bokeh examples.

Week 7-8: Voice Recognition Integration

Task 6: Voice Recording and Processing

Dependencies: Completion of Display and Interaction.

Subtasks:

1. Planning:
 - Define voice recognition requirements.
2. Designing:
 - Plan integration with UI.
3. Developing:
 - Implement voice recording with sounddevice.
 - Integrate speech_recognition for processing.
4. Testing:
 - Verify accuracy and reliability of voice processing.
5. Evaluating:
 - Ensure integration meets user needs.

Resources: sounddevice, speech_recognition libraries.

Week 9: Controller Logic and Integration

Task 7: Event Handling and Data Synchronization

Dependencies: Completion of Voice Recording and Processing.

Subtasks:

1. Planning:
 - Define interaction logic.
2. Designing:
 - Map out event handlers and synchronization.
3. Developing:
 - Write event handlers for UI actions.
4. Testing:
 - Verify end-to-end functionality.
5. Evaluating:
 - Ensure smooth operation and integration.

Resources: Python, Tkinter, SQLite.

Task 8: Voice Recognition Integration

Dependencies: Completion of Event Handling and Data Synchronization.

Subtasks:

1. Planning:
 - Define interaction logic for voice recognition.
2. Designing:
 - Plan voice recognition integration.
3. Developing:
 - Implement voice recognition logic.
4. Testing:
 - Verify functionality.
5. Evaluating:
 - Ensure voice recognition meets requirements.

Resources: sounddevice, speech_recognition libraries.

Week 10: Error Handling and Validation**Task 9: Input Validation**

Dependencies: Completion of Voice Recognition Integration.

Subtasks:

1. Planning:
 - Identify validation requirements.
2. Designing:
 - Create validation rules.
3. Developing:
 - Implement validation logic.
4. Testing:
 - Verify handling of invalid inputs.
5. Evaluating:
 - Ensure reliability and user feedback.

Resources: Python error handling techniques.

Task 10: Error Handling

Dependencies: Completion of Input Validation.

Subtasks:

1. Planning:
 - Identify error handling requirements.
2. Designing:
 - Create error messages.
3. Developing:

- Implement error handling logic.
- 4. Testing:
 - Verify handling of errors.
- 5. Evaluating:
 - Ensure reliability and user feedback.

Resources: Python error handling techniques.

Week 11-12: Testing, Evaluation, and Deployment

Task 11: Testing

Dependencies: Completion of Error Handling.

Subtasks:

1. Planning:
 - Develop testing strategy.
2. Designing:
 - Create test cases.
3. Developing:
 - Perform unit, integration, and user testing.
4. Testing:
 - Conduct thorough testing.
5. Evaluating:
 - Address any issues identified during testing.

Resources: Testing tools.

Task 12: Deployment

Dependencies: Completion of Testing.

Subtasks:

1. Planning:
 - Develop deployment plan.
2. Designing:
 - Create deployment package.
3. Developing:
 - Package the application for distribution.
4. Testing:
 - Verify deployment process.
5. Evaluating:
 - Ensure all dependencies are included and installation instructions are clear.

Resources: Deployment guides.

Task 13: Evaluation

Dependencies: Completion of Deployment.

Subtasks:

1. Planning:
 - Plan evaluation process.
2. Designing:
 - Create feedback forms.
3. Developing:
 - Gather user feedback.
4. Testing:
 - Evaluate feedback.
5. Evaluating:
 - Make necessary improvements.

Resources: User feedback tools.

System Models

Criterion C - System Overview

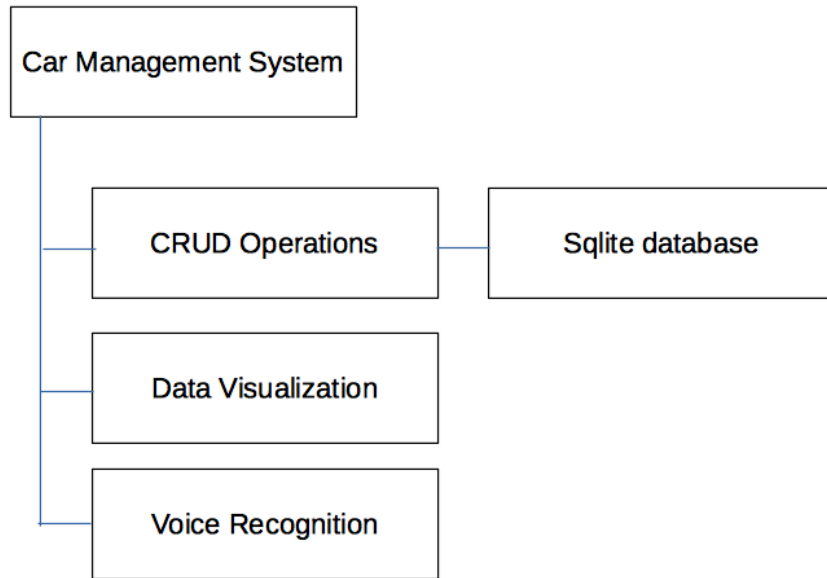


Image 1: Overall Structure

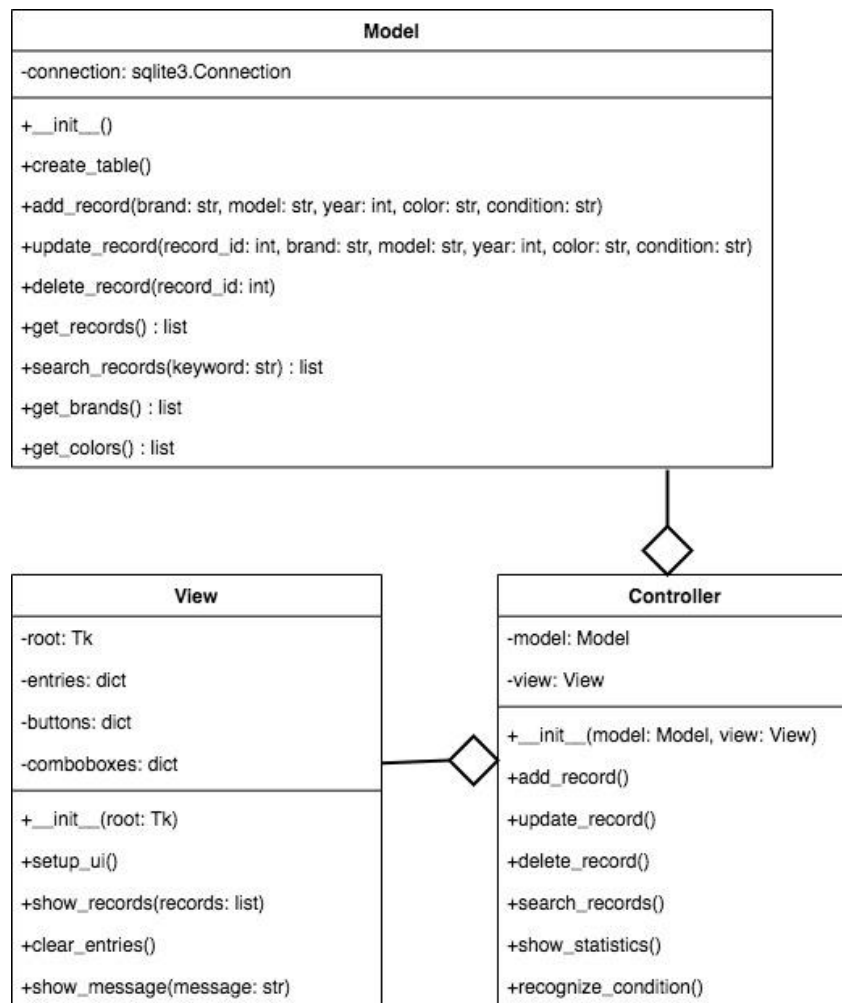
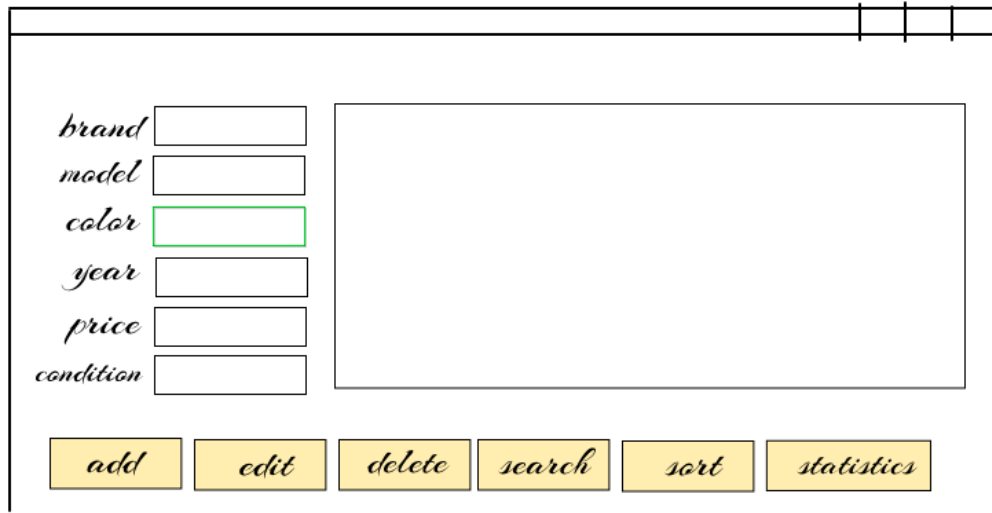


Image 2: UML Use Case Diagram

Database Schema. Sqlite.

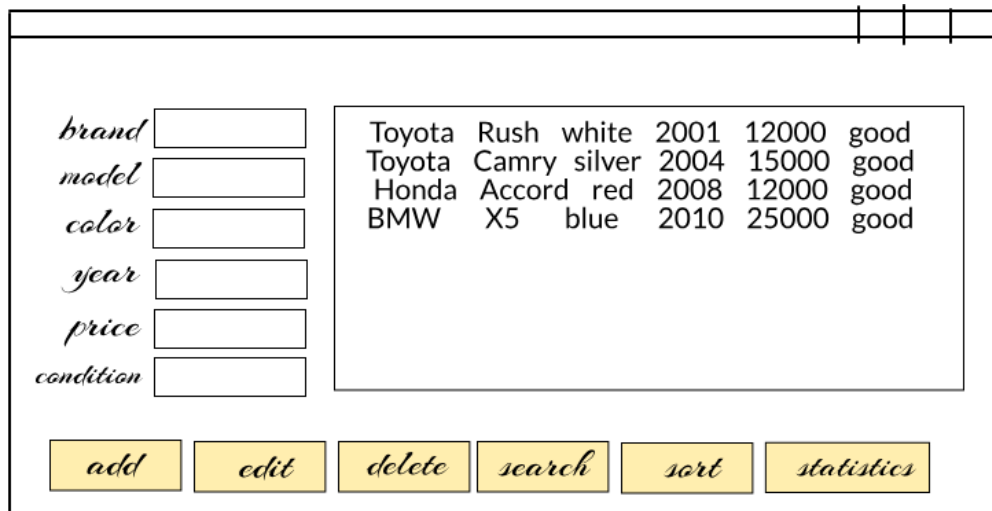
- **id** (INTEGER PRIMARY KEY AUTOINCREMENT): A unique identifier for each car record.
- **brand** (TEXT): The brand of the car.
- **model** (TEXT): The model of the car.
- **color** (TEXT): The color of the car.
- **year** (INTEGER): The year the car was manufactured.
- **price** (REAL): The price of the car.
- **condition** (TEXT): The condition of the car.

“Condition” will be entered using Voice Recognition



A GUI window with a title bar and three window control buttons. On the left, there are six labels: *brand*, *model*, *color*, *year*, *price*, and *condition*, each followed by a text input field. The *color* field has a green border. To the right of these fields is a large empty rectangular box. At the bottom, there are six yellow buttons with black text: *add*, *edit*, *delete*, *search*, *sort*, and *statistics*.

Image 3: GUI prototype with no records



The same GUI window as in Image 3, but the large rectangular box now contains a table with 4 records. The labels and buttons remain the same.

Toyota	Rush	white	2001	12000	good
Toyota	Camry	silver	2004	15000	good
Honda	Accord	red	2008	12000	good
BMW	X5	blue	2010	25000	good

Image 4: GUI prototype with 4 records

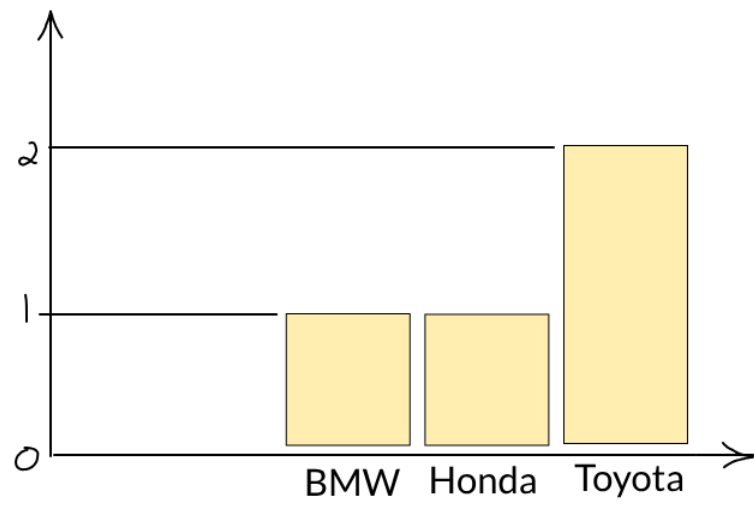


Image 5: Data Visualization based on 4 records

Algorithms as components in a system model

SQL Statements to create the table:

```
CREATE TABLE IF NOT EXISTS cars (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    brand TEXT NOT NULL,
    model TEXT NOT NULL,
    color TEXT NOT NULL,
    year INTEGER NOT NULL,
    price REAL NOT NULL,
    condition TEXT NOT NULL
);
```

SQL Statements to add a record:

```
INSERT INTO cars (brand, model, color, year, price, condition)
VALUES (?, ?, ?, ?, ?, ?),
(brand, model, color, year, price, condition)
```

SQL Statements to delete a record:

```
DELETE FROM cars WHERE id = ?, (record_id,)
```

SQL Statements to display all records:

```
SELECT * FROM cars
```

SQL Statements to count cars grouped by brand:

```
"SELECT brand, COUNT(*) FROM cars GROUP BY brand
```

Voice recognition for the Car Condition

```
FUNCTION recognize_condition():
    OPEN audio stream using sounddevice
    RECORD audio for 5 seconds
    CONVERT recorded audio to AudioData object
    TRY:
        RECOGNIZE speech using Google Web Speech API
        UPDATE 'condition' text field in the GUI with recognized text
    CATCH UnknownValueError:
        DISPLAY error message "Sorry, I did not understand that."
```

Data Visualization

```
FUNCTION show_statistics():  
    CONNECT to 'cars.db'  
    EXECUTE SQL command to get the count of cars by brand:  
        SELECT brand, COUNT(*) FROM cars GROUP BY brand  
    STORE result in 'brand_counts' variable  
    CLOSE database connection  
    GENERATE a bar plot using Bokeh:  
        X-axis: brands  
        Y-axis: counts  
    DISPLAY plot in a new window
```

Data Validation

```
FUNCTION validate_inputs(brand, model, color, year, price, condition):  
    IF NOT brand OR NOT model OR NOT color OR NOT year OR NOT price  
    OR NOT condition:  
        RETURN False  
    IF NOT year.isdigit() OR NOT price.replace('.', '', 1).isdigit():  
        RETURN False  
    RETURN True
```

Testing Strategies

Success Criteria	Testing Strategy	Expected Result
1. Validate data	Add, Edit, Delete Search records with blank fields and non numerical values for Price and Year to confirm that these functions will not be performed	Non numerical values for Price and Year will result in an error message when adding/editing records. Empty fields will result in an error message when adding, editing, deleting, searching records
2. Add a car record	Fill out all fields, press the Add button and verify that a new record is successfully displayed in the table	A record with valid fields will be stored in a database and be displayed in the table
3. Edit existing car record	Fill out all fields using the same ID number for the record that should be edited and verify that the record has been updated	A record with valid fields and the same ID number will overwrite the existing record and will be displayed in the table
4. Delete a car record	Select a record and press the delete button. Verify that the record is no longer displayed in the table	If a record is not selected an error message will be displayed. Otherwise a selected record will be deleted from the database and will not longer be displayed in the table
5. Search for a particular car	Enter a keyword for one of the fields and press the search button. A record containing that keyword will be displayed in the table	If no keywords are entered an error message will be displayed. Otherwise if a record that contains a the keyword or a part of it, will be displayed in the table
6. Sort cars by brand name, price and year	Press on the chosen sorting option, and verify that records are organized by name, or price or by year	If sort by brand is chosen, then records will be displayed in alphabetical order of brand name. If sort by price is chosen, then records will be displayed from lowest to highest price If sort by year is chosen, then records will be displayed from oldest to newest cars
7. Display statistical data using a graph	Press on statistics button and verify that the graph is displayed representing existing car records	The bar graph will display columns for each car brand. The high of the column represent the number of cars of that specific brand

8. Have a voice recognition capability	Press on the Speak button, and say an expression representing the condition of the car.	If the spoken audio expression is recognized, then the field will be filled in with the expression. Otherwise an error message is displayed.
--	---	--

Criterion D - Development

MVC Pattern. Appendix.pdf, page 1,4,6



controller.py



main.py



model.py



view.py

MVC pattern promotes a well-structured and organized codebase, which is easier to develop, maintain, and scale. It ensures that the application is modular, with clear separation between data handling, user interface, and application logic, leading to more efficient and parallel development.

Model (model.py): Handles all database operations, such as adding new car records, updating existing records, deleting records, and retrieving records for display or statistics.

View (view.py): Provides the graphical user interface for the application, including forms for inputting car details, buttons for various actions (Add, Update, Delete, Search, etc.), and displays the records fetched from the Model.

Controller (controller.py): Receives user inputs from the View, validates and processes these inputs, calls appropriate methods in the Model to interact with the database, and updates the View with the results of these operations. It also handles the voice recognition feature for the car condition field.

The testing strategy does not test this pattern

Data Validation. Appendix.pdf page 6

```
def validate_entry(self, data):
    # Validate entry data
    try:
        year = int(data[3])
        price = float(data[4])
    except ValueError:
        return False
    return all(data[:3]) and all(data[5:]) and year > 0 and price > 0
```

By validating inputs before adding, editing, searching, and deleting records, the application ensures that the data remains consistent, correct, and safe from malicious inputs. Checking if year is within a valid range ensures that the data reflects realistic values. It also ensures that price is a positive number and prevents incorrect financial data from being stored.

The testing strategy is effective and successfully rejects the invalid data particularly when the Year and Price is not a digit, or a negative number.

Adding a record to a database. Appendix.pdf page 4

```
def add_record(self, brand, model, color, year, price, condition):
    with self.conn:
        # Add a new record to the database
        self.conn.execute(
            "INSERT INTO cars (brand, model, color, year, price, condition) VALUES (?, ?, ?, ?, ?, ?)",
            (brand, model, color, year, price, condition)
        )
```

The `add_record` method is suitable for adding new car records to the database due to its simplicity, security, and maintainability. Using placeholders (?) for the values in the SQL statement ensures that the values are properly escaped, preventing SQL injection attacks and ensuring the integrity and safety of the database operations. The use of a context manager (`with self.conn:`) ensures that transactions are automatically committed if no exceptions occur. If an exception is raised, the transaction is automatically rolled back, preventing partial or corrupt data entries.

The testing strategy is effective and successfully adds a new record to the database if all fields are filled in and validated. Otherwise an error message is displayed.

Editing Records. Appendix.pdf page 5

```
def update_record(self, record_id, brand, model, color, year, price, condition):
    with self.conn:
        # Update a record in the database by ID
        self.conn.execute(
            """
            UPDATE cars
            SET brand = ?, model = ?, color = ?, year = ?, price = ?, condition = ?
            WHERE id = ?
            """,
            (brand, model, color, year, price, condition, record_id)
        )
```

The SQL statement updates the columns (brand, model, color, year, price, condition) with new values for the record where the id matches `record_id`.

The placeholders (?) in the SQL statement are used to safely insert the values, preventing SQL injection attacks.

The tuple (brand, model, color, year, price, condition, record_id) contains the actual values to be updated in the corresponding columns and the record ID for the WHERE clause.

The method directly interacts with the database using efficient SQL commands. The use of parameterized queries enhances performance by allowing the database engine to optimize the execution plan for the query.

The testing strategy is effective and successfully edits a record if all fields are filled in and validated. Otherwise an error message is displayed.

Deleting Records. Appendix.pdf page 4

```
def delete_record(self, record_id):
    with self.conn:
        # Delete a record from the database by ID
        self.conn.execute("DELETE FROM cars WHERE id = ?", (record_id,))
```

The method `delete_record` takes a single parameter, `record_id`, which is the unique identifier for the car record that needs to be deleted. The placeholder (?) in the SQL statement is used to safely insert the value of `record_id`, preventing SQL injection attacks.

The tuple `(record_id, )` contains the actual value of the record ID to be deleted.

The method is easy to maintain and extend. If additional criteria for deletion are needed in the future, the method can be updated with minimal changes to the SQL statement and parameters.

The testing strategy is effective and successfully deletes a record, if it is selected, otherwise an error message is displayed.

Searching algorithms. Appendix.pdf page 5

```
def search_records(self, **kwargs):
    query = "SELECT * FROM cars WHERE "
    query += " AND ".join([f"{k} LIKE ?" for k in kwargs.keys()])
    values = [f"%{v}%" for v in kwargs.values()]
    # Search for records based on criteria
    cursor = self.conn.execute(query, values)
    return cursor.fetchall()
```

The method `search_records` accepts a variable number of keyword arguments (`**kwargs`). Each keyword represents a column in the `cars` table, and its corresponding value represents the search criteria for that column.

`query = "SELECT * FROM cars WHERE "` initializes the base SQL query to select all columns from the `cars` table.

`query += " AND ".join([f"{k} LIKE ?" for k in kwargs.keys()])`

dynamically constructs the WHERE clause of the SQL query based on the provided keyword arguments. It iterates over the keys (column names) of the `kwargs` dictionary and creates a LIKE ? condition for each column, joining them with AND.

The method is highly flexible, allowing for searching based on any combination of columns and values. It can handle various search criteria dynamically without needing separate methods for each combination of search conditions.

The testing strategy is effective and successfully searches for records based on any search criteria.

Sorting algorithms. Appendix.pdf page 5

```
def sort_by(self, field):
    # Sort records by a specific field
    cursor = self.conn.execute(f"SELECT * FROM cars ORDER BY {field}")
    return cursor.fetchall()
```

The `sort_by` method is designed to sort the car records in the SQLite database based on a specified field. The method `sort_by` accepts a single parameter `field`, which specifies the column name by which the records should be sorted. The `ORDER BY` clause in SQL is used to sort the result set by one or more columns. The field name is directly inserted into the query string using an f-string. The method allows for sorting by any column in the `cars` table, making it versatile and adaptable to different sorting requirements.

The testing strategy is effective and successfully sorts existing records by brand name, price and year, assuming the database contains records.

Display Statistical Data using a graph. Appendix.pdf page 9

```
def show_statistics(self):
    # Show statistics using Bokeh
    data = self.model.get_car_counts_by_brand()
    if data:
        brands, counts = zip(*data)
        source = ColumnDataSource(data=dict(brands=brands, counts=counts))
        p = figure(x_range=brands, height=350, title="Car Counts by Brand", toolbar_location=None, tools="")
        p.vbar(x='brands', top='counts', width=0.9, source=source)
        p.xgrid.grid_line_color = None
        p.y_range.start = 0
        output_file("car_counts.html")
        show(p)
    else:
        messagebox.showinfo("Statistics", "No data available for statistics.")
```

The `show_statistics` method is designed to display statistics about car counts by brand using Bokeh for visualization.

The algorithm includes a check for data availability, ensuring that users are informed if there is no data to display. This improves the user experience by providing clear feedback in cases where the visualization cannot be generated. `output_file("car_counts.html")`: This specifies that the plot should be saved to an HTML file named "car_counts.html".

`show(p)`: This displays the plot in a web browser.

The testing strategy is effective and successfully displays a graph based on car brands and their quantity, assuming the database contains some records.

Voice Recognition. Appendix.pdf page 8

```

def recognize_condition(self):
    # Use voice recognition to fill the condition field
    recognizer = sr.Recognizer()
    with sd.InputStream(samplerate=16000, channels=1, dtype='int16') as stream:
        print("Listening...")
        audio = stream.read(int(16000 * 5)) # Record 5 seconds of audio
        audio_data = np.frombuffer(audio[0], dtype=np.int16)
        audio_file = io.BytesIO(audio_data.tobytes())
        audio_data = sr.AudioData(audio_file.getvalue(), 16000, 2)
    try:
        text = recognizer.recognize_google(audio_data)
        print(f"You said: {text}")
        self.view.entries['condition'].delete(0, tk.END) # Clear the text field
        self.view.entries['condition'].insert(0, text) # Insert recognized text
    except sr.UnknownValueError:
        print("Sorry, I did not understand that.")
    except sr.RequestError:
        print("Sorry, there was an error with the speech recognition service.")

```

The recognize_condition method is designed to use voice recognition to fill in the "condition" field, which can be faster and more convenient than typing, especially for users who are not comfortable with typing or have accessibility needs. It uses the speech_recognition and sounddevice libraries to capture and recognize voice input for a Tkinter-based application's "condition" field. It initializes a Recognizer, records 5 seconds of audio using sounddevice, converts the audio to a suitable format, and then uses Google's Web Speech API to transcribe the audio into text.

The testing strategy is effective and successfully records and recognizes the speech. An error message is displayed if the speech cannot be recognized.

Words: 1060

Criterion E - Evaluation

Meeting Success Criteria

1. Validate data

Achieved. Record fields are validated to make sure no record with invalid data is added to a database. All text fields should contain data, and numerical fields should contain only positive numbers so that the record considers as valid

2. Add a car record

Achieved. Records with valid fields are added to the database. Each record is automatically assigned an ID number which is automatically incremented and displayed in the table. Invalid records will be rejected and will not be added to the database.

3. Edit existing car record

Achieved. Fields of an existing record are successfully updated if the record is selected (so that the ID number is identified) and new valid data is entered. The existing record is updated, by overwriting with a new record.

4. Delete a car record

Achieved. Selected record is successfully deleted from the database. A confirmation dialog box is also displayed.

5. Search for a particular car

Achieved. The search function successfully searches and displays records that match one or multiple parameters. If there are no records matching parameters, no output is displayed

6. Sort cars by brand name, price and year

Achieved. The sorting function successfully arranges records based on chosen sorting order such as by brand name (from A to Z), by price (from cheapest to expensive) or by year (from oldest to newest).

7. Display statistical data using a graph

Achieved. A graph is displayed visualizing the number of cars of a particular brand. The height of a column corresponds to the number of cars. No column is displayed if there is no car of that brand.

8. Have a voice recognition capability

Achieved. It successfully records the speech for 5 seconds, recognizes it, and converts it as a text. The precision a lot depends on the clarity of the speech and the background noise.

Suggesting improvements

The solution functions effectively as a standalone application. However, to accommodate broader user access and facilitate future growth, transitioning to a web-based platform could be beneficial. This would support business expansion and enable multiple second-hand car dealerships in various locations to utilize the system.

Currently, the application only tracks available cars and lacks functionality for monitoring sold vehicles. Incorporating features to record sales and maintain a database of car owners would provide a more comprehensive overview.

Additionally, to ensure that only authorized personnel can access the application, implementing a user authentication system is essential. This system would enable users to create secure usernames and passwords, thereby safeguarding access and protecting sensitive information.

Word count 376